MOTION OS

Compose Motion & Pixels

Choosing the right API · live demos in the app

This deck is one updateTransition. Every flourish names its API.

GOAL

Leave knowing which API

1. Which API for which problem — not a catalog dump
2. When Canvas / custom Layout is actually worth it
3. Shared performance rules we can ship with

AGENDA

Five beats, then Q&A

1. Animation decision tree + live proof
2. Graphics & Layout tree + live proof
3. Performance checklist
4. Apply to our apps
5. Q&A

RESTRAINT

When not to animate

- Prefer system / Material defaults first
- Motion must serve hierarchy or feedback
- If it fights readability or a11y — cut it
- Games and Canvas are rare in product UI

Custom is for moments that sell the product.

ANIMATION

Need motion?

Tap a branch we will prove

- ├─ `Appear / disappear?` → `AnimatedVisibility`
- ├─ `Swap content / screens?` → `AnimatedContent`
- ├─ `Many props, one state?` → `updateTransition`
- ├─ `Imperative / interruptible?` → `Animatable`
- ├─ `Gesture / organic settle?` → `spring()`
- ├─ `Authored “story” timing?` → `keyframes`
- ├─ `List ↔ detail continuity?` → `SharedTransition`
- ├─ `Per-frame draw / game?` → `withFrameNanos + Canvas`

Live app: tappable branches jump to proof. PDF: pattern only.

SPECS

Intent, not decoration

`tween`

Directed, timed · enter / morph

`spring`

Physical · drag release, playful

`keyframes`

Hand-authored · overshoot, pauses

`infinite`

Loop · loaders (sparingly)

CATALOG

Animation APIs — what they do

APPEAR & SWAP

`AnimatedVisibility`	Show or hide with Enter / Exit
`AnimatedContent`	Swap screens or content (A → B)
`Crossfade`	Fade-only content swap
`animateContentSize`	Animate size when children change

DRIVE VALUES

`animate*AsState`	One property toward one target
`updateTransition`	Many properties, one discrete state
`Animatable`	Imperative: snap, stop, sequence, cancel
`rememberInfiniteTransition`	Looping values — loaders, ambient
`animateDecay`	Fling from remaining velocity

SPECS · INTENT

`tween`	Directed, timed — enter, morph, structure
`spring`	Physics settle — drag release, playful tap
`keyframes`	Authored story — race, pause, overshoot
`snap`	Jump with no interpolation
`repeatable`	Play N times, or forever

CONTINUITY

`SharedTransitionLayout`	Scope wrapping both destinations
`sharedElement`	The same visual object travels
`sharedBounds`	Bounds morph; content inside can differ

PER-FRAME

`withFrameNanos`	Vsync clock — simulation, games, orbits

Not every function in androidx.compose.animation — the set we choose from. Tree picks; table names.

PROOF

Morphing Action Button

- One state. Width, color, glyph stay in lockstep.
- Tween the structure. Spring the glyph.
- Pay / save / sync in our apps.

`updateTransition` `animateDp` `PathMeasure` `graphicsLayer`

```
updateTransition(state)
animateDp { Idle → 280.dp; Loading → 66.dp }
textAlpha  delayMillis = 120  // structure leads
```

LIVE IN MOTION OS

Idle → Loading → Done

Not interactive in PDF. Run the app on a tablet (shortest side ≥ 600 dp) and open Morphing Action Button.

showcase/components/MorphingActionButton.kt

PSEUDOCODE

updateTransition — morphing button

```
one state  →  many properties


updateTransition(buttonState)
  width   ← tween      Idle / Loading / Done
  color   ← tween
  label   ← tween + 120ms delay   // structure leads
  glyph   ← spring                // micro-interaction
```

Full source in Android Studio

app/.../showcase/components/MorphingActionButton.kt

PROOF

Keyframes Theme Wipe

- Tap to wipe light ↔ dark.

Animatable · keyframes · clipRect

```
keyframes {
  0.90f at 520  using cinematic
  overshoot at 700
  target at 850
}
```

LIVE IN MOTION OS

Race · overshoot · settle

Animation drives clipRect — a drawing primitive, not a layout property. Open Keyframes Theme Wipe in the app.

`showcase/KeyframesThemeWipeDemo.kt`

PSEUDOCODE

keyframes + clipRect wipe

```
Animatable fraction   0 → 1

keyframes:
  race to 90%     @ 520ms
  overshoot       @ 700ms
  settle          @ 850ms

clipRect(width × fraction)   // draw, not layout
```

Full source in Android Studio

`app/.../showcase/KeyframesThemeWipeDemo.kt`

PROOF

Shared Element Gallery

- The card travels — it doesn't swap.
- Keys: swatch-id, title-id.
- Compose's answer to hero transitions.

`SharedTransitionLayout` `sharedElement` `AnimatedContent`

```
SharedTransitionLayout {
  Modifier.sharedElement(
    rememberSharedContentState("swatch-$id")
  )
}
```

LIVE IN MOTION OS

The card travels

Stable unique keys. Don’t share huge subtrees. Test back stack in real nav — open Shared Element Gallery in the app.

`showcase/SharedElementGalleryDemo.kt`

GRAPHICS

Draw or place?

Custom drawing or layout?

```
├── Tint / shape / clip only?            → Modifier / Shape
├── Charts, gauges, game art?            → Canvas + DrawScope
├── Children in custom positions?        → custom Layout
├── Deferred / dependent measure?        → SubcomposeLayout
```

Canvas is not a layout system. Layout is not a drawing API.

DRAW

Canvas Pulse Chart

- Designer gives you a custom chart? This is where you land.
- One fraction. Many properties.
- Same cinematic pattern as UI morphs.

Canvas | Path | Animatable | stagger

```
val t = remember { Animatable(0f) }
Canvas { drawPath(area, brush) }
localT = ((t - i * 0.06f) / 0.7f)
```

LIVE IN MOTION OS

Grid · area · bars

All DrawScope. One Animatable(0→1); bars stagger locally. Open Canvas Pulse Chart in the app.

showcase/CanvasPulseChartDemo.kt

LAYOUT

Fan Deck

- Layout places. Canvas draws the ring.
- zIndex = frontness. graphicsLayer sells depth.
- Swipe / tap — spring settle. Product: wallet, stories, picker.

Layout placeRelative zIndex graphicsLayer spring

```
val a = PI/2 + 2π * (i - selected) / n
p.placeRelative(x, y, zIndex = frontness)
// graphicsLayer { rotationY = …; scale = depth }
```

LIVE IN MOTION OS

Measure, then place

No Row/Column trigonometry hacks. Open Fan Deck in the app and swipe.

showcase/OrbitalMenuLayoutDemo.kt

PSEUDOCODE

custom Layout — fan deck

```
Layout:
  measure each child
  place on ellipse(selected)
  zIndex = frontness          // who is in front

graphicsLayer:  rotationY, scale   // depth, not placement
Canvas:         draw the ring

spring settle after swipe
```

Full source in Android Studio

app/.../showcase/OrbitalMenuLayoutDemo.kt

INTENSITY

Same toolbox. Different intensity.

A · CINEMATIC UI

Aurora Unlock

Hold the core. Animatable, springs, shared clock.

B · GAME LOOP

Neon Rush

withFrameNanos racer. Same Canvas primitives. Not default product UI.

SHIP

Performance checklist

1 graphicsLayer / draw / offset { } — not layout every frame

2 Cache Path / paints in hot 60fps loops

3 One shared clock for coordinated effects

4 Don’t animate a state read in every parent

5 Shared elements: stable keys, small regions

6 Shaders / AGSL: API guards + fallbacks

OUR APPS

Fill this with the room

MOMENT	API
Primary CTA async	`Morphing button / updateTransition`
List → detail	`Shared elements`
Theme / onboarding flourish	`keyframes (sparingly)`
Analytics widget	`Canvas chart`
Radial / fan picker	`custom Layout (if design requires)`
Everything else	`Material + Visibility / Content`

Canvas / Layout only when Material + standard animation can’t express the design.

LEAVE-BEHIND

The lab is this repo

1. Compose Lab → app/.../showcase/
2. Handout → docs/compose-lab-handout.md
3. This PDF deck → docs/talk/index.html
4. github.com/abdul-wahab-9d/MotionOS

No tablet? Phone layout is a tap-through lab.

Q & A

Your motion bugs, please

Where have we fought motion jank?

Any design ask that forced Canvas prematurely?
Who wants a follow-up on Nav + shared elements?

LAYOUT

Measure children, then placeRelative — that’s custom layout from scratch. Canvas is for pixels, not for placing children.

ANIMATION

Visibility you know. Custom means Animatable + a spec. Story timing is keyframes. Physics is spring. Continuity across screens is shared elements.

THANK YOU

TEAM LEAD

Hammad Nawaz

and the team

Animatable · spring · keyframes · sharedElement · Canvas · Layout

HOW TO RUN

Motion OS — tablet = deck

1. Clone the repo and run the app in Android Studio
2. Tablet or Pixel Tablet AVD (shortest side ≥ 600 dp) opens this talk
3. Phone opens Compose Lab — the demo list
4. Clicker: volume / D-pad / space · notes: N

Demos in this PDF are stills. The motion lives in the app.